



01/17(토) - 2주차

담당자	이예빈
유형	주간

이번주 해야할 일

- ☐ ppt 작성
- ☐ 최종 제안서 작성

- ai 튜터에 대한
 - 선행 자료
 - 어떤식으로 구현할 수 있을지
 - 어떤 정보를 이용해서 어떤 로직을 짜야하는지 등등
- 이예빈
 - 지능형 튜터(intelligence tutoring system : ITS)
 - <https://goldmandarin.tistory.com/entry/지능형-튜터링-시스템Intelligence-Tutoring-System-ITS>
 - 1970년대부터 연구가 시작되었으며,수학이나 물리와 같이 잘 정의된 지식의 구조를 갖는 영역에 대해 단계별 학습을 할 수 있도록 활용됨
 - 지능형 튜터링 시스템은 도메인 모델, 교육자 모델, 학습자 모델로 구성됨
 - 도메인 모델 : 학습 주체에 대한 지식을 뜻함
 - 교육자 모델 : 어떻게 가르칠 것인가
 - 학습자 모델 : 학습자들의 가정된 지식 상태를 나타냄 → 끊임없이 학생들의 학습 상태를 확인하고 데이터를 갱신하고 업데이트해서 개선해 나감 → 이게 있어서 개인의 수준에 따라 맞춤형 학습을 제공할 수 있음
 - 장점

● 반렌반렌((VanLehn, 2011)에 따르면 ITS가 대면방식의 일대일 교수법에 비해 -0.19의 효과 크기를 보이지만, 강의식 수업과 비교하였을때에는 ITS가 0.47 수준의 효과 크기를 보인다 하였습니다.
 - KT(정오답 예측), SP(점수 예측), RS(문제 추천)의 연구가 존재 그 중 Knowledge Tracing이라는 지식 추적(KT)란
 - 특정 학생의 지난 교육 기록을 활용해 아직 풀지 않은 문제에 대한 정오답을 예측하는 Task
 - 예를 들어 산타 토익과 같은 문제 풀이 앱을 사용할 때, 기존 사용자가 푼 문제 1~6를 학습해 사용자의 학습 수준을 유추하고 이를 활용하여 문제 78910의 정오답을 예측
 - 1994년에 발표된 Bayesian Knowledge Tracing(BKT)이 ai 세대 이전에 가장 대표적인 방법론 → 개념 이해(knowledge component/skill)에 대한 사용자의 수준을 추정하고 같은 개념에 대한 정오답 확률을 예측
 - 딥러닝은 과거의 기록으로 미래의 행동을 예측하는 기계학습이 탁월한 성능을 보이는 대표적인 구조인데 이걸 적용한 걸 DKT(Deep Knowledge Tracing)이라고 보면 됨
 - 사용자를 분석하고 static한 feature들을 뽑아내는 것이 아닌 문제 풀이 기록을 순차 데이터로 활용하여 자연어처리에서도 자주 활용되는 RNN 기법을 적용한 것
 - ai tutor의 학습 방법
 - 링글은 chat gpt에 영어 학습에 최적화된 프롬프트를 입력해서 ai tutor로서의 역할을 수행
 - 문법 및 표현 교정, 발음 분석 및 교정, 개인화된 학습 자료 제공
 - ai tutor를 이용한 선행 사례

- <https://www.ringleplus.com/ko/student/landing/blog/ai-tutor>
 - 말하기 연습을 하면서 영어 회화 능력을 키워줌
- 전체 파이프라인
 - 수집 : 시도, 요청, 힌트 열람, 제출, 성공/실패
 - 분석 : 규칙/패턴 기반 분류기(초반 MVP), 숙련도 추정, 정책(힌트/다음 문제 추천)
 - 개입 : 힌트 단계/강도 조절, 다음 문제 추천, 학습 경로 재구성
- ITS
 - 도메인 : 문제/스킬
 - 학습자 : 사용자 상태
 - 튜터링 : 힌트/추천 정책
- KT
 - 학습자의 상호작용 로그(정오답, 시도, 시간 등)으로 미스터리 확률을 추정하고 다음 개입(힌트나 추가 문제)를 결정 → BKT 사용
- 어떤 정보를 수집 해야 되는지
 - ai 튜터가 의사결정할 수 있는 최소 로그
 - 풀이시도
 - 특징 단위
 - 페이로드의 특징 → xss의 단서, sqli의 단서 등등
 - 시도 흐름 특징 → 같은 유형 반복 횟수, 성공 직전의 패턴 변화
 - 스킬 매핑
 - 어떤 문제에 어떤 스킬이 필요한지?
- 로직
 - 페이로드/행동 분류기 → 현재 문제의 기대 스킬/유형과 일치하는지 방향성을 잡아주는 느낌
 - 숙련도 추정
 - 합리적인 변형을 시도했는지 보고 만약 전혀 다른 유형을 반복하면 개념이 잘못 매칭된 형태임을 ㄱ추정
 - 튜터 정책
 - 힌트 제공 시점
 - ~분 이상
 - ~회 이상
 - 기대유형과 시도 유형의 불일치
 - 힌트 강도
 - 개념 키워드/용어
 - 취약점 위치 힌트 → 입력 벡터, 싱크 지점
 - 다음에 시도할 한 단계 → 인코딩 한 번 적용
 - 다음 문제 추천 → 숙련도를 보고 그에 해당하는 보강하거나 높이는 문제 추천
 -
- 김다인
 - ITS
 - : AI 지능형 튜터의 전통적 정의는 ITS (Intelligent Tutoring System) 에서 출발
 - 4대 구성요소
 - Domain Model (도메인 모델)
 - 가르칠 지식 구조

- 예: CTF라면
 - Web → SQLi, XSS, CSRF
 - Pwn → BOF, ROP
 - Crypto → RSA, XOR, Padding Oracle
 - Student Model (학습자 모델)
 - 학습자가 무엇을 알고 있고, 무엇을 모르는지
 - 실력, 오개념, 숙련도
 - Tutoring Model (튜터링 전략)
 - 언제 힌트를 줄지
 - 얼마나 직접적으로 알려줄지
 - 문제를 바꿀지 말지
 - Interface Model (상호작용)
 - 텍스트, 챗, 시각화, 코드, 로그
- ⇒ AI 튜터 구현 = ITS를 게임 속 CTF 환경에 이식
- But, 기존 ITS는 정답이 명확하고 풀이 경로가 제한적이지만 CTF는 풀이경로가 무수히 많고 시행착오 자체가 학습으로 이어져 틀렸다는 정의가 애매함
- ⇒ (행동 기반 모델링 + 과정 중심 피드백) 핵심
- 최신 흐름
 - 기존 Rule-Based ITS X
 - LLM + 로그 기반 추론형 튜터
 - AI 튜터의 기능
 - 사고 단계 진단
 - 지금 막힌 이유 분석
 - Ex. 개념 부족, 도구 사용 미숙, 문제 이해 부족
 - 힌트 레벨 조절
 - 메타 학습
 - Ex.
 - 항상 입력 검증 단계를 건너뛰는 경향이 있다.
 - Crypto 문제에서 수식 세우는 걸 회피하는 패턴이 보인다
 - AI 튜터가 수집해야 하는 정보
 - 필수 수집 로그
 - 문제 관련
 - Ex. 문제 난이도, 핵심 관련 등
 - 플레이어 행동 로그
 - Ex. 시도 로그, 상호 작용 등
 - 실패 유형 로그
 - 튜터링 로직
 - 상태 기반 튜터링
 - 튜터에 입력할 내용
 - 현재 문제 정보
 - 플레이어 능력 벡터
 - 최근 행동 로그

- 4

◦ 참고하면 좋을 자료

- TryHackMe 의 echo : 직접 한번 사용해보면 좋을 듯
 - 사용자의 학습 과정을 관찰하고 다음에 진행할 단계를 안내 → 실시간 도움 제공
 - 완료한 후에는 Echo가 사용자의 성과를 검토하고 맞춤형 피드백을 제공
 - 현재 베타 버전으로 무료/Jr Penetration Tester 과정의 모든 강의실에서 이용 가능하며, TryHackMe의 모든 주요 학습 과정에 순차적으로 도입될 예정
 - <https://tryhackme.com/echo>



- LLM 기반 지능형 튜터링(ITS)+프롬프트 엔지니어링+RAG 사례
 - GPT-3.5 계열 LLM + RAG(벡터DB) + 이중 레이어 프롬프트(고정 템플릿 + 동적 사용자 메타데이터)로 개인 맞춤 튜터를 구현
 - 정해진 콘텐츠(문제·풀이 예시 등)를 벗어나지 않도록 LLM을 감싸는 구조를 응용하면 좋을 것 같습니다
 - <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453719/>
- ITS(Intelligent tutoring system) 키워드 더 알아보면 도움이 될 듯!!
 - <https://arxiv.org/abs/2305.13272>
 - <https://arxiv.org/html/2507.18882v1>
 - <https://sathee.iitk.ac.in/adaptive-difficulty-system/>

◦ 구현에 필요한 데이터

- 할 것 : 단계별 힌트, 부분 풀이, 오류 분석, 다음 문제 제안
- 문제 메타데이터
 - problem_id, category(ai/web/pwn/crypto/forensic 등), 난이도 레벨, 공식 풀이 단계(step1~n), 의도한 핵심 개념 태그들(sqli, xss, prompt injection 등).
- 사용자 시도 로그(한 문제 기준)
 - user_id, problem_id, 시도 시각, 정답 여부, 제출 플래그/입력 일부, 사용 명령/ 스크립트
 - 풀이 시간(문제 오픈~정답 제출까지), 힌트 사용 횟수/단계, 중간 포기 여부(로그인 시점을 기준으로?)
- 풀이 기록(누적)
 - 최근 N문제 성공률, 카테고리별 성공률/실패율, 반복 오답 유형

⇒ 난이도 추천용 feature vector 생성, 학습자 상태 요약(e.g. '이 사용자는 web easy는 90% 성공, SQLi 관련 실패 다수' 등) 생성

◦ LLM 튜터 엔진

- API 기반 상용 LLM + RAG 구조(각 문제의 풀이, 참고 자료 등을 벡터 DB에 임베딩)
 - 해당 문제에 대한 풀이/힌트 문서만 retrieve 해서 LLM에 넣도록 제한
- 정책
 - 프롬프트에 "정답 플래그 문자열, 완전 익스플로잇 코드를 직접 출력하지 말 것" 같은 룰 추가
 - 단계별 규칙 설계
 - 첫 번째 힌트: 개념 키워드/취약점 유형 언급
 - 두 번째 힌트: 취약할 가능성이 큰 코드 영역/기능을 제시
 - 세 번째 힌트: payload 구조 예시, pseudo code 수준까지 허용 등

- 한지선

- 선행 자료

CodeSignal의 AI tutor cosmo → <https://codesignal.com/cosmo/>

<https://www.forbes.com/sites/rayravaglia/2025/08/20/codesignal-cosmo-and-the-future-of-practice-first-learning/>

IDE와 결합된 맥락 인식형 AI 튜터

→ IDE 내부에 내장되어 있어 현재 문제가 무엇인지, 사용자가 작성한 코드가 어떤지, 실행 결과 및 에러를 실시간으로 평가하고 도움 제공

<https://www.hackthebox.ai/>

<https://www.hackthebox.com/blog/ai-assistant>

Hack the Box의 AI 기반 학습 도우미

→ 우리가 구현하려는 것 처럼 AI가 사용자의 질문, 현재 문제 정보 등을 가지고 힌트 같은 걸 제공하는 AI인 것 같음.

- 어떤식으로 구현할 수 있을지

학생이 문제를 풀면서 지식 상태가 어떻게 변하는지 확인

- DKT (Deep Knowledge Tracing)

이 학생이 다음번에 이 문제를 맞힐 확률 값 제공

학생이 물어보는 문제나, 학생한테 다음으로 풀 문제를 추천할 때 사용하면 좋지 않을까 싶음.

확률 값에 따라서 힌트의 깊이나, 문제 추천 난이도를 다르게 하는 방식으로

→ 모델 학습 코드 예시도 존재함. 대신 학습 시키려면 문제 풀이 별 정오답 기록 같은 데이터가 필요한데 이걸 우리 위게임에서 어떻게 구할지 생각해 봐야 할 것 같음.

<https://stanford.edu/~cpiech/bio/papers/deepKnowledgeTracing.pdf>

- 시스템

- 문제 풀이 화면 옆에 채팅 형식의 오버레이 형태의 채팅 인터페이스로 응답을 주고 받을 수 있게 설계하면 어떨지
 - 문제의 정답, 의도, 취약한 코드 부분을 벡터 DB에 저장해두고, AI가 이를 참고해서 힌트 제공
 - 사용자의 트래픽을 로깅 형태로 전달 받을 수 있게 해야 함.

- 필요한 정보 및 로직

- 정보

- 문제 풀이 상태
오답률, 걸린 시간 등을 종합해 나온 학생을 수준을 파악할 데이터
 - 문제 풀이 기록
학생이 어떤 시도를 하고 있었는지, 어떤 방식으로 문제를 풀려고 했는지
Ex) 페이로드나 툴 이용했으면 그 결과가 어떻게, 어떤 명령어를 사용했는지
 - 시간 기록
어느 부분에서 얼마나 오래 머물러 있었는지
 - 참고 자료 조회 여부
→ 화면에서 벗어난 시간 같은 거 추적하면 알 수 있지 않을까

- 튜터 로직

- 1. 학생의 질문 + 학생 수준을 종합해 응답 수준 결정
 - 2. 바로 답을 주지 않게 프롬프트 엔지니어링? 같은 거 추가
→ 유도해서 풀 수 있게끔

문제, 학생에 점수를 부여해서 학생이 문제를 맞추거나 틀리면 그에 맞는 점수 하락이나 상승
사용자가 계속 틀리는 문제에 맞춰서 관련한 문제를 추천하는 로직

데이터를 어떻게 할지.. 규칙 기반이나 가상의 페르소나 같은 거 만들어서 학습시킬 수 있으면 괜찮을 것 같음.

데이터 수집은 웹 문제 같은 경우 프록시 하나 두고 사용자가 페이지로 날릴 때만 수집, AI는 API 호출될 때만 텍스트 데이터 수집해야
될 것 같음.

한 번에 모아놔다가 문제 하나 풀면 학습하게 하는 느낌으로

-
- 챗봇으로 진행?
 - 데이터 수집?
 - ai tutor의 역할
 - 대화형 → 챗봇 형태
 -
 - Q&A(출력 형태 제한)
 -
 - **힌트나 다음 문제 추천 + a**
 - 예빈, 다인
 - ㅁㅁ

역할 분담 (5명)

문서 3명에 / 발표 & ppt 2명

ppt 제작

- ㅁㅁ, 이예빈

문서 작성 → 빨간 글씨 부분

- 박민영
- 이예빈
- 한지선
- 김민서
- 김다인

발표

- 김다인

-
- 프로젝트 개요
 - 김다인
 - 특징 및 기능
 - 김다인
 - 기능적 요구사항 (완)
 - 한지선
 - **UI**
 - **이예빈**
 - 대블록 + 소블록
 - 김다인
 - flow chart
 - 한지선
 - use case 명세 (완)
 - 한지선
 - use case diagram (완)

- 한지선
- system sequence diagram
 - 박민영
- class diagram
 - 이예빈
 - 김민서
- sequential diagram
 - 이예빈
 - 김민서
- DB
 - 김민서
- 시나리오? 작동하는거
 - 한지선
- WBS (완)
 - 박민영
- 관련 기술
 - 한지선
- 기대효과
 - 박민영

일요일까지 기능 초안을 작성해서 일요일 기능 확정하고 **일요일 밤 10시**

이거 바탕으로 지선이가 먼저 use 명세랑 등등 수정해주면 → 월요일(서면)

24일 토요일 밤 11시? → 피드백 및 수정

26일까지 완성하고 (ppt 초안) → 서면으로 받기 (합치기 : 이예빈)

27일에 ppt 최종 완성하고 발표 점검하고 28일에 가서 하면 늦어? → 27일 오후 1시?

28일에 못 오는 사람 : 박민영